

Software - Architecture, Design, Test, and...

Troels Mortensen

June 4, 2026

Contents

1	3-Layered Architecture	1
1.0.1	From Two Layers to Three	1
1.1	Why We Use Data-Centric Three Layers	1
1.1.1	Logical Layers vs Physical Tiers	2
1.1.2	When Data-Centric Three Layers Fit	2
1.1.3	Transitive Dependency and Layer Monopolies	3
1.1.4	Relaxed Read Paths	3
1.1.5	A Fowler Variation (Not This Chapter's Anchor)	4
1.1.6	The Entities Refinement	4
1.1.7	Classic Build Order	4
1.2	Step 1: Database and Data Access Layer	4
1.2.1	Schema and Entity Mapping	5
1.2.2	DAOs and Unit of Work	5
1.2.3	Database Constraints and Business Rules	10
1.3	Step 2: Business Logic Layer	10
1.3.1	Transaction Script Services	10
1.3.2	Business Service Example in Java	11
1.4	Step 3: Presentation Layer	11
1.4.1	Presentation Mapping Example in Java	12
1.4.2	Other Driving Entry Points	12
1.4.3	Validation, Errors, and Leaky Boundaries	13
1.5	When the Business Layer Grows	13
1.5.1	Breaking Cycles Between Services	13
1.5.2	Sibling Helpers Inside the Business Layer	13
1.5.3	Dependencies Inside the Application Layer	14
1.5.4	Layer Packages vs Feature Packages	14
1.6	Evolution: DTO Module	15
1.6.1	DTO Example in Java	15
1.6.2	Who Maps What	15
1.7	Recommended Package and Module Layout	15
1.8	Assembling Data for Views	17
1.8.1	Approach A: Data-Access-Level Assembly	18
1.8.2	Approach B: Business-Layer Assembly	19
1.8.3	Same View, Different Trade-offs	21
1.8.4	SQL Pitfalls When the Business Layer Orchestrates Reads	21
1.8.5	CQRS Perspective for Read Models	21
1.8.6	Decision Guidance	22
1.9	Testing Strategy Across Layers	22
1.10	When Persistence Becomes Infrastructure	22
1.10.1	Case Trigger from Abyssal Watch	22
1.10.2	Why Persistence Can Be Too Narrow	22

1.10.3	Recommended Evolution Path	22
1.10.4	Mission Alert After Detection	23
1.10.5	Trade-offs and Guardrails	24
1.11	Schema Change Ripples	24
1.11.1	A Worked Example: Renaming a Column	24
1.12	Practical Trade-offs and Common Mistakes	25
1.12.1	Layered and Hexagonal: Practical Relationship	25
1.12.2	Closing Thoughts	26

Todo list

IN-REF: write layer definition chapter (what-is-architecture)	1
DIAGRAM: three-layer stack inside a monolith boundary: presentation, application, persistence, database	2
DIAGRAM: strict write path vs relaxed read exception (presentation to application to persistence vs presentation to read DAO)	3
IN-REF: add reference to Transaction Script pattern chapter when written	11
DIAGRAM: write use case flow: presentation controller to ConfirmKaijuDetectionService to DAOs to database	11
IN-REF: add reference to Object Parameter Pattern chapter when written	13
DIAGRAM: application sub-package DAG: service to shared, factory, assembler; no service-to-service edges	14
DIAGRAM: module dependency arrows: presentation to application to persistence; config wires implementations	16
IN-REF: add reference to Modules chapter when written	16
DIAGRAM: Mission Kaiju Briefing: DAL join query path vs BLL multi-DAO assembler path	18
DIAGRAM: ConfirmKaijuDetectionService to persistence DAOs and to infrastructure.messaging alert adapter	24

Chapter 1

3-Layered Architecture

This chapter teaches *data-centric three-layered architecture*: a classic way to structure business applications where the relational database schema is the anchor and software is built in layers on top of it.[3, 4] We separate the system into *presentation*, *business logic*, and *data access*, so each part has a clear responsibility. In Java packages we often use the names **presentation**, **application** (for the business logic layer), and **persistence** (for the data access layer).

We apply this style in a **logical layered monolith**: one deployable application where packages or modules express layers. A formal definition of *layer* appears in a later part of this book. Physical deployment patterns are out of scope here.

In this chapter, we use the **Abyssal Watch** case study with two running examples: the **write** use case **Confirm First Kaiju Detection** (SCN-02) and the **read** use case **View Mission Kaiju Briefing**. If you read the 2-layered chapter first, you may have introduced a standalone **entities** module shared by presentation and persistence. In the classic three-layer style, table-shaped persistence entities live *inside* the data access layer instead; we explain that refinement in Section 1.1.6.

In the course that accompanies this book, you create database tables and write SQL yourself. The code snippets therefore use **persistence entities** (plain Java classes aligned to tables) and **JDBC DAOs** with hand-written **PreparedStatement** queries—not an ORM.

IN-REF:
write layer
definition
chapter
(what-is-
architecture)

1.0.1 From Two Layers to Three

Chapter ?? kept business logic in presentation controllers (JavaFX or Web API) while persistence stored domain entities from a shared **entities** module. Three layers add an explicit **application** (business logic) layer: controllers stay thin, and services such as **ConfirmKaijuDetectionService** (Code snippet 1.7) own rules and transaction sequencing. Persistence entities move under **persistence** rather than a top-level **entities** module shared with presentation—see Section 1.1.6.

1.1 Why We Use Data-Centric Three Layers

The main benefit of this style is separation of concerns around **data management and entity flow**: how records are captured at the top, processed in the middle, and written to tables at the bottom.[4] We isolate user interaction in **presentation**, procedural rules and transaction sequencing in the business logic layer (**application**), and SQL and entity mapping in the data access layer (**persistence**).

The default dependency direction is:

- **presentation -> business logic -> data access -> database**

In practice:

- **presentation** translates user actions into calls the business layer understands and formats results for humans or APIs,
- the business logic layer sequences reads and writes, enforces rules, and opens transaction boundaries,
- the data access layer converts between table rows and code objects; the database remains the source of truth for stored state.

A useful mental model:[3]

- **Presentation** — the counter clerk: accepts requests and shows results, without deciding business outcomes,
- **Business logic** — the kitchen manager: coordinates work, applies rules, and decides what gets stored,
- **Data access** — the supply pantry: fetches and puts away stock according to storage layout, without interpreting policy.

1.1.1 Logical Layers vs Physical Tiers

Do not confuse the **logical layers** in this chapter with a **physical three-tier deployment**. Logical layers are package or module responsibilities inside one deployable application: presentation, business logic, and data access code living in the same process, with dependency rules between them. A physical three-tier setup places presentation, application, and database on separate machines or processes (for example a web server, an app server, and a database server). The names overlap, but the concerns differ: here we teach structure inside the monolith; how you deploy processes is a separate topic. Figure 1.1 summarizes the logical stack we build in this chapter.

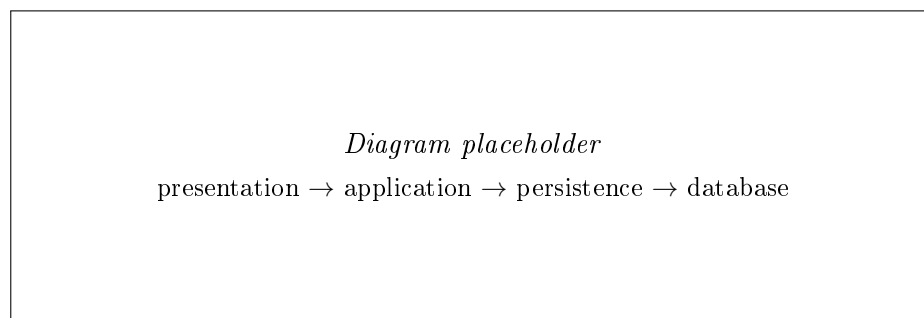


DIAGRAM:
three-layer
stack inside
a monolith
boundary:
presentation,
application,
persistence,
database

Figure 1.1: Logical three-layer stack inside one deployable application (diagram to be added).

1.1.2 When Data-Centric Three Layers Fit

This style is a strong default when:

- the system is CRUD-heavy and transactional, with a stable relational schema at the center,
- the team ships features bottom-up from tables and DAOs,
- onboarding benefits from predictable layer placement rules.

Signs you may be outgrowing the baseline (without abandoning layering):

- business service classes become oversized and hard to test in isolation,

- provider-shaped DAO interfaces fight the vocabulary your use cases need,
- relaxed read shortcuts spread until policy is unclear,
- public API contracts suffer on every schema rename.

1.1.3 Transitive Dependency and Layer Monopolies

Transitive dependency rule: presentation must not skip the business logic layer to call the data access layer directly, except in deliberate relaxed read paths (Section 1.1.4). Skipping layers hides dependencies and makes schema changes hard to contain.[3]

Monopoly rules:

- the business logic layer owns business decisions and transaction sequencing,
- the data access layer owns SQL, entity mapping, and persistence mechanics—not business rules,
- presentation owns interaction and formatting—not persistence or policy.

Fight leakage *downward* (rules buried in stored procedures) and *upward* (smart UI widgets or controllers that orchestrate storage).

1.1.4 Relaxed Read Paths

Default: read flows go through the business layer when policy, reuse across channels, or invariant checks matter.

Exception: read-only, presentational lookups with *no* business decision—for example static labels, reference data for dropdowns, or data whose rules were already enforced on a prior write. Allowed shapes include a thin read facade in **application** or, in the strictest relaxation, **presentation** calling a specialized read-only DAO or query object. There are still no writes and no transaction orchestration in presentation.

Risks include schema coupling, duplicated read logic, and policy drift when screens grow rules later. Mitigate with read projections owned by data access (or **dtos**) and revisit the path when business logic appears.

Confirm First Kaiju Detection is **not** a relaxed read. For **View Mission Kaiju Briefing**, the thinnest data-access-level assembly variant may use a relaxed path; see Section 1.8. Figure 1.2 contrasts the default write path with the relaxed read exception.

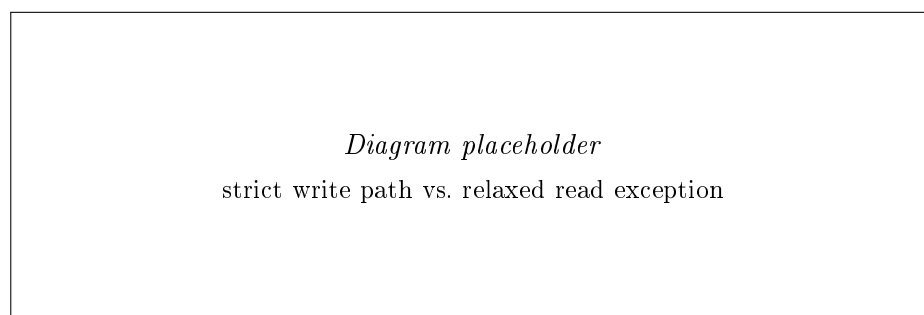


DIAGRAM:
strict write
path vs re-
laxed read ex-
ception (pre-
sentation to
application to
persistence vs
presentation
to read DAO)

Figure 1.2: Default write path through the business layer vs. relaxed read paths (diagram to be added).

1.1.5 A Fowler Variation (Not This Chapter’s Anchor)

Martin Fowler describes a layered enterprise model with presentation, *domain*, and data source.[1] The domain layer can hold rich behavior that is not tied one-to-one to tables. That variation is common and valuable, but it is not the data-centric workflow we teach here. This chapter stays **schema-first and bottom-up**; a later chapter on hexagonal architecture explores **behavior-first** design with explicit ports and adapters.

1.1.6 The Entities Refinement

We first describe the stack as:

- presentation -> business logic -> data access -> database

For clarity we often refine the bottom of the stack to:

- presentation -> business logic -> data access -> entities -> database

Entities are not a fourth deployable tier and presentation must not call them directly. They are **table-shaped persistence entities**—plain Java classes whose fields align with columns—that the data access layer maps to and from the database using SQL you write. The business layer manipulates those types *through* data access APIs such as KaijuDao (Code snippet 1.2), not by reaching past the data access layer into storage.

In packages, entities belong *inside* data access, for example `persistence.entities`, not as a top-level sibling module at the solution root (contrast Chapter ??, where a separate `entities` module was a stepping stone for small systems).

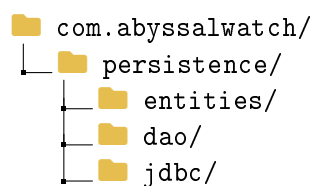


Figure 1.3: Entity types nested under the data access layer, closest to the database.

1.1.7 Classic Build Order

When a team adds a feature in this style, work usually flows **bottom-up**:[3]

1. design tables and build the data access layer (mapping, DAOs),
2. write business services that use those data shapes,
3. expose services through UI or API controllers.

1.2 Step 1: Database and Data Access Layer

We start with the database because the schema is the blueprint. For **Confirm First Kaiju Detection** we need at least a `breach` table and a `kaiju` table with a foreign key to the origin breach (INV-04). For **View Mission Kaiju Briefing** (Section 1.8) we also need a `mission` table linked to the tracked Kaiju.

1.2.1 Schema and Entity Mapping

A simplified relational sketch:

- `breach(id, ...)`
- `kaiju(id, codename, classification, origin_breach_id, mission_id, ...)` with `origin_breach_id` referencing `breach(id)` and `mission_id` referencing `mission(id)` when assigned
- `mission(id, status, ...)` for briefing state (for example `Briefed`, `Deployed`)

We map these tables to entity classes in `persistence.entities`; Code snippet 1.1 shows the shape for `kaiju`. Field names mirror the columns in the `CREATE TABLE` scripts you write. Close alignment between tables and entity classes is a **feature** in data-centric systems: it keeps transactional CRUD predictable.[3]

```
1 // persistence.entities - table-shaped entity (no ORM)
2 public class KaijuEntity
3 {
4     private String id;
5     private String codename;
6     private KaijuClassification classification;
7     private String originBreachId;
8     // constructors, getters, setters omitted
9 }
```

Code snippet 1.1: KaijuEntity table-shaped persistence entity

1.2.2 DAOs and Unit of Work

The data access layer exposes DAOs defined from storage needs. The business layer depends on these abstractions; SQL and JDBC details stay in `persistence.jdbc`.

Contracts are often *provider-shaped*: interfaces mirror what storage already offers (`findById`, `save`) because the data access layer defines them from tables first. That is mockable and effective for data-centric work.

A table-shaped method fits when the schema supports it: `KaijuDao.findByIdByMissionId(...)` in Code snippet 1.2 because `kaiju.mission_id` links a Kaiju to a mission. A use-case-shaped need such as *find all sensors within radius of a breach* does not map cleanly to one table method—the DAO starts to fight the vocabulary the use case needs. That friction is a signal to evolve toward core-owned ports (Chapter ??), not a reason to abandon DAOs for straightforward CRUD.

```
1 public interface BreachDao
2 {
3
4     Optional<BreachEntity> findById(BreachId breachId);
5
6 }
7
8 public interface KaijuDao
9 {
10
11     void save(KaijuEntity kaiju);
12
13     Optional<KaijuEntity> findByMissionId(MissionId missionId);
14
15 }
16
17 public interface UnitOfWork
18 {
19
20     <T> T execute(Supplier<T> action);
21
22 }
```

Code snippet 1.2: BreachDao, KaijuDao, and UnitOfWork contracts

The JDBC implementations below follow the same pattern: obtain a `Connection` from `ConnectionProvider.current()`, run parameterized SQL, and translate `SQLException` into `PersistenceException`.

```
1 public final class JdbcBreachDao implements BreachDao
2 {
3     private final ConnectionProvider connections;
4
5     @Override
6     public Optional<BreachEntity> findById(BreachId breachId)
7     {
8         String sql = "SELECT id FROM breach WHERE id = ?";
9         Connection conn = connections.current();
10        try (PreparedStatement ps = conn.prepareStatement(sql))
11        {
12            ps.setString(1, breachId.value());
13            try (ResultSet rs = ps.executeQuery())
14            {
15                if (!rs.next())
16                {
17                    return Optional.empty();
18                }
19                return Optional.of(
20                    new BreachEntity(rs.getString("id"))
21                );
22            }
23        }
24        catch (SQLException ex)
25        {
26            throw new PersistenceException(
27                "Failed to load breach",
28                ex
29            );
30        }
31    }
32 }
```

Code snippet 1.3: JdbcBreachDao.findById

JdbcBreachDao.findById is a read-only lookup: a single SELECT with one bind parameter, returning Optional.empty() when no row exists.

```
1 public final class JdbcKaijuDao implements KaijuDao
2 {
3     private final ConnectionProvider connections;
4
5     @Override
6     public void save(KaijuEntity kaiju)
7     {
8         String sql = ""
9             INSERT INTO kaiju (
10                 id, codename, classification, origin_breach_id
11             )
12             VALUES (?, ?, ?, ?)
13             "";
14         Connection conn = connections.current();
15         // Do not close conn: UnitOfWork owns the lifecycle.
16         try (PreparedStatement ps = conn.prepareStatement(sql))
17         {
18             ps.setString(1, kaiju.getId());
19             ps.setString(2, kaiju.getCodename());
20             ps.setString(
21                 3,
22                 kaiju.getClassification().name()
23             );
24             ps.setString(4, kaiju.getOriginBreachId());
25             ps.executeUpdate();
26         }
27         catch (SQLException ex)
28         {
29             throw new PersistenceException(
30                 "Failed to save kaiju",
31                 ex
32             );
33         }
34     }
35 }
```

Code snippet 1.4: JdbcKaijuDao.save

JdbcKaijuDao.save performs an INSERT whose columns mirror KaijuEntity. The comment on connections.current() matters: the DAO must not close the connection while UnitOfWork still owns the transaction.

```
1 public final class JdbcUnitOfWork implements UnitOfWork
2 {
3     private final ConnectionProvider connections;
4
5     @Override
6     public <T> T execute(Supplier<T> action)
7     {
8         try (Connection conn = connections.open())
9         {
10             conn.setAutoCommit(false);
11             try
12             {
13                 connections.bind(conn);
14                 T result = action.get();
15                 conn.commit();
16                 return result;
17             }
18             catch (RuntimeException ex)
19             {
20                 conn.rollback();
21                 throw ex;
22             }
23             finally
24             {
25                 connections.unbind();
26             }
27         }
28         catch (SQLException ex)
29         {
30             throw new PersistenceException(
31                 "Transaction failed",
32                 ex
33             );
34         }
35     }
36 }
```

Code snippet 1.5: JdbcUnitOfWork.execute

JdbcUnitOfWork.execute opens the connection, disables auto-commit, binds it for nested DAO calls, and commits or rolls back on success or failure.

```
1 public interface ConnectionProvider
2 {
3
4     Connection open();
5
6     void bind(Connection connection);
7
8     Connection current();
9
10    void unbind();
11
12 }
```

Code snippet 1.6: ConnectionProvider

In Code snippet 1.6, `ConnectionProvider` supplies the JDBC `Connection` for the current unit of work: `open` and `bind` for `JdbcUnitOfWork`, `current` for DAOs in the same transaction. DAO implementations must not close a connection obtained from `current()` while the unit of work is still active.

1.2.3 Database Constraints and Business Rules

Use the database for **structural integrity** and the business layer for **policy**:

- **Database:** foreign keys (for example INV-04 via `origin_breach_id`), NOT NULL, unique indexes—the safety net that still holds under races and alternate callers.
- **Business layer:** mission state gates, classification rules, and user-facing validation messages—the place that expresses intent.
- **Avoid:** triggers or stored procedures that encode policy (Section 1.1.3).

For **Confirm First Kaiju Detection**, the service in Code snippet 1.7 checks that the origin breach exists before insert; the foreign key on `origin_breach_id` still protects integrity if two requests race. Prefer friendly errors from the service; let the database enforce what must never be violated.

1.3 Step 2: Business Logic Layer

With data structures in place, we write business services that load and save through the data access layer and enforce procedural rules. For **Confirm First Kaiju Detection**, the service validates that the origin breach exists, constructs a new Kaiju record shape, and persists it inside a transaction.

The business layer knows entity shapes because the data access layer exposed them (Code snippet 1.1)—not because we modeled the use case in a vacuum first.

1.3.1 Transaction Script Services

Table-shaped types in `persistence.entities` are intentionally **anemic** in this style—data containers with little or no behavior. Rules and sequencing live in services (and helpers such as **factory** or **assembler**), not in entity methods. That differs from Fowler’s domain-layer variation in Section 1.1.5, where behavior may live in a model that is not tied one-to-one to tables.

1.3.2 Business Service Example in Java

```

1 public final class ConfirmKaijuDetectionService
2 {
3     private final BreachDao breachDao;
4     private final KaijuDao kaijuDao;
5     private final UnitOfWork unitOfWork;
6
7     public KaijuId confirmDetection(
8         String codename,
9         KaijuClassification classification,
10        BreachId originBreachId
11    )
12    {
13        return unitOfWork.execute(() ->
14        {
15            breachDao.findById(originBreachId)
16                .orElseThrow();
17
18            KaijuEntity kaiju = new KaijuEntity();
19            kaiju.setId(KaijuId.generate().value());
20            kaiju.setCodename(codename);
21            kaiju.setClassification(classification);
22            kaiju.setOriginBreachId(originBreachId.value());
23
24            kaijuDao.save(kaiju);
25            return new KaijuId(kaiju.getId());
26        });
27    }
28 }

```

Code snippet 1.7: ConfirmKaijuDetectionService transaction script

Code snippet 1.7 follows Fowler's **Transaction Script** pattern: a procedural script over the DAOs in Code snippet 1.2, with one use case per method.[1]

1.4 Step 3: Presentation Layer

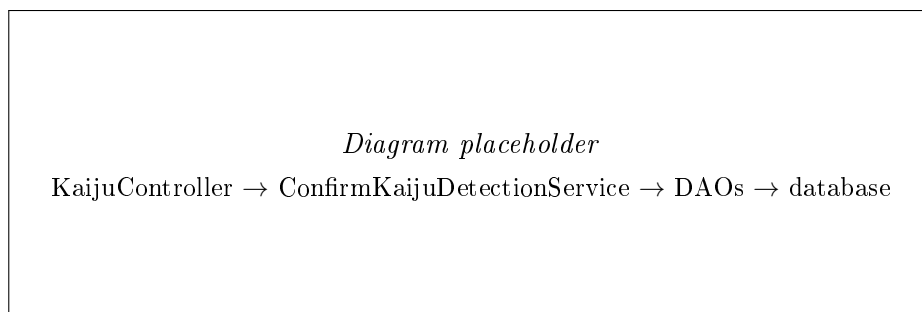


Figure 1.4: Confirm First Kaiju Detection write flow through the layers (diagram to be added).

Finally we expose business services to users. Figure 1.4 summarizes the write path for **Confirm First Kaiju Detection**. The presentation layer stays thin: parse input, call the business service, format the response. It must not embed business rules or call DAOs directly.

IN-REF: add reference to Transaction Script pattern chapter when written

DIAGRAM: write use case flow: presentation controller to ConfirmKaijuDetectionService to DAOs to database

The examples use **Spring MVC** annotations (`@RestController`, `@PostMapping`) as one common way to expose HTTP in Java. The architectural rule is thin adapters that delegate to services—not Spring itself. The same structure applies with other web frameworks or with non-HTTP entry points (Section 1.4.2).

Typed request and response records can live in `presentation.web` at first, or in a dedicated `dtos` module when API contracts need stronger isolation (Section 1.6).

1.4.1 Presentation Mapping Example in Java

Code snippet 1.8 shows a thin REST adapter: it maps HTTP input to service parameters and delegates to Code snippet 1.7.

```
1 public record ConfirmKaijuDetectionRequest(  
2     String codename,  
3     KaijuClassification classification,  
4     BreachId originBreachId  
5 ) {}  
6  
7 public record ConfirmKaijuDetectionResponse(String kaijuId) {}  
8  
9 @RestController  
10 @RequestMapping("/kaiju-detections")  
11 public class KaijuController  
12 {  
13     private final ConfirmKaijuDetectionService service;  
14  
15     @PostMapping  
16     public ResponseEntity<ConfirmKaijuDetectionResponse> confirm(  
17         @RequestBody ConfirmKaijuDetectionRequest request  
18     )  
19     {  
20         KaijuId id = service.confirmDetection(  
21             request.codename(),  
22             request.classification(),  
23             request.originBreachId()  
24         );  
25  
26         return ResponseEntity.accepted()  
27             .body(new ConfirmKaijuDetectionResponse(id.value()));  
28     }  
29 }
```

Code snippet 1.8: KaijuController REST presentation adapter

1.4.2 Other Driving Entry Points

Presentation is any **driving** entry point that translates outside interaction into calls the business layer understands. In this chapter we use a REST controller (Code snippet 1.8) because three-layered systems in the course are usually exposed through HTTP. The same business service (Code snippet 1.7) could equally be invoked from a CLI handler, a scheduled job, or a message consumer: keep the adapter thin, delegate to the service, and do not call DAOs directly on writes. Business rules stay in `application`; only the transport changes.

1.4.3 Validation, Errors, and Leaky Boundaries

Each layer has a distinct job when input fails or storage rejects a change:

- **Presentation** parses and validates *shape*: required fields, formats, and ranges suitable for HTTP or UI. It maps outcomes to status codes or messages the client sees. It does not decide business policy.
- **Business logic** enforces *policy*: for example that the origin breach exists before insert (Section 1.2.3). Prefer clear application exceptions (`UnknownBreachException`) over generic failures.
- **Data access** runs SQL and translates `SQLException` into persistence-layer failures. Map those to business-meaningful errors at the boundary before they reach presentation, rather than leaking raw SQL state upward.

Leaky abstractions appear when layers share the wrong types: `SQLException` or `ResultSet` in a controller, HTTP request types in a service, or persistence entities returned directly from REST without a DTO evolution path (Section 1.6). The `orElseThrow()` calls in Code snippets 1.7 and 1.8 are placeholders; production code should throw domain- or application-level errors the presentation layer can translate consistently.

1.5 When the Business Layer Grows

A small system can live in a flat `application.service` package. `Confirm First Kaiju Detection` fits that shape. When more use cases arrive, two problems appear: **service-to-service cycles** and **responsibilities that do not fit a single service class**.

1.5.1 Breaking Cycles Between Services

A textbook pattern outside our case study: `CustomerService` and `OrderService` each grow features that need the other (`customer with order history` vs. `order with customer details`). Peer service calls create a circular dependency.

The same pattern appears in `Abyssal Watch`: `MissionService` (briefing with Kaiju summary) and `KaijuTrackingService` (track with mission context) may tempt each other into cross-calls. Extract shared linkage logic into `application.shared`—for example `MissionKaijuContext`—that both services use. `shared` must not depend on `service`, and services must not depend on each other.

1.5.2 Sibling Helpers Inside the Business Layer

When logic is real business concern but clutters a service, use sibling packages by *kind* of work: `factory` (construction and invariants), `assembler` (combining data for a use case or view), and `shared` (cross-use-case linkage). A `KaijuFactory` can enforce INV-04; `MissionBriefingAssembler` can build the briefing view in Code snippet 1.11; triangulation fusion for SCN-03 may live in another `assembler`. Services still own transaction boundaries and DAO calls.

Optional evolution: when a service method such as `confirmDetection` accumulates many parameters, Fowler’s *Introduce Parameter Object* refactor groups them into one type.[2] That is an alternative to growing a long parameter list; defer it until the signature is genuinely hard to read.

Do not confuse `application.shared` with `persistence.entities`: the latter holds table-shaped persistence entities; `shared` holds cross-use-case business logic. Do not confuse either with `dtos` (Section 1.6), which are presentation-facing contracts.

IN-REF: add reference to Object Parameter Pattern chapter when written

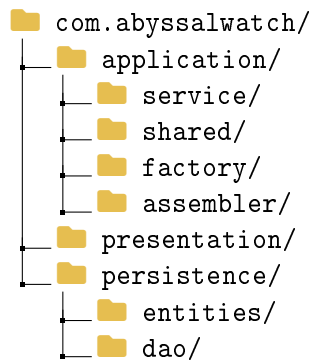


Figure 1.5: Business layer sub-packages when the flat `service` package is no longer enough.

Introduce sub-packages only when the flat layout is hard to navigate—not on day one.

1.5.3 Dependencies Inside the Application Layer

Inside `application`, dependencies should form a directed acyclic graph:

- `service` may depend on `shared`, `factory`, and `assembler`,
- `shared`, `factory`, and `assembler` must not depend on `service`,
- services must not depend on each other—extract shared logic (for example `MissionKaijuContext`) instead.

Keep types in the right bucket:

- `persistence.entities` — table-shaped persistence entities,
- `application.shared` — cross-use-case business logic,
- `dtos` — presentation-facing contracts (Section 1.6).

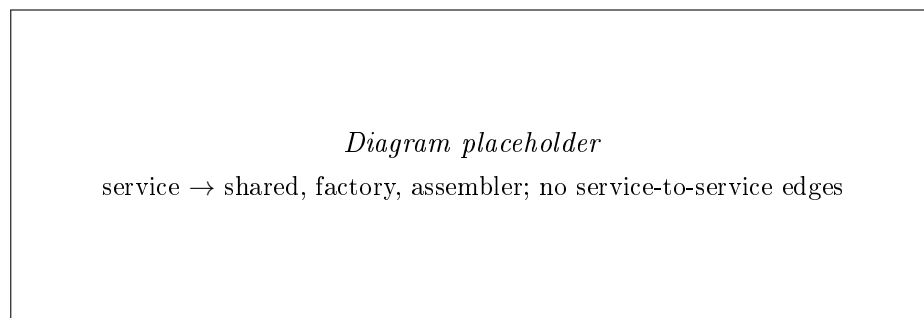


DIAGRAM:
application
sub-package
DAG: service
to shared,
factory, as-
sembler; no
service-to-
service edges

Figure 1.6: Directed dependencies inside the `application` layer (diagram to be added).

1.5.4 Layer Packages vs Feature Packages

The layouts in this chapter use **layer packages**—`presentation`, `application`, `persistence`—because they match the bottom-up, schema-first workflow students follow: design tables, write JDBC DAOs (Code snippets 1.3–1.6), add services, then controllers. Layer placement is easy to teach and review.

Feature packages (or vertical slices) group everything for one capability together, for example `kaiju.detection` containing controller, service, and DAO for that flow. That helps when one

feature changes often and the team wants related code in one place. The trade-off is that technical concerns scatter: SQL, HTTP mapping, and policy for the same story live in different roots unless you discipline sub-structure.

A practical hybrid keeps layer roots but adds feature subfolders under `application.service` when a domain area grows (for example `service.kaiju`, `service.mission`). Hexagonal architecture discusses feature-folder trade-offs in more depth (Chapter ??); here layer packages remain the default for data-centric three-layer systems.

1.6 Evolution: DTO Module

A dedicated `dtos` module is **not** required for the classic baseline. Add it when HTTP or UI contracts must stay stable while entity or service signatures change—versioning, hiding persistence fields, or shaping responses per use case.

Mapping adds code, but it decouples presentation from table-shaped types exposed by the business and data layers.

1.6.1 DTO Example in Java

When API contracts must stay stable while service or entity signatures change, move types such as `ConfirmKaijuDetectionRequest` and `ConfirmKaijuDetectionResponse` from `presentation.web` into a dedicated `dtos` module (see Code snippet 1.8 for the baseline shape).

1.6.2 Who Maps What

Mapping belongs at the boundary that introduces a new representation:

- **Presentation** maps HTTP or UI input to service parameters or `dtos`, and maps results to responses (Code snippet 1.8).
- **Application** maps entity shapes to use-case inputs; `assembler` helpers compose read models when using business-layer assembly.
- **Data access** maps entities to the database with SQL; read projections for view assembly live here in Approach A (Section 1.8).

Avoid passing the same shape through every layer by habit. Add mapping when a transport or storage technology forces a distinct type.

1.7 Recommended Package and Module Layout

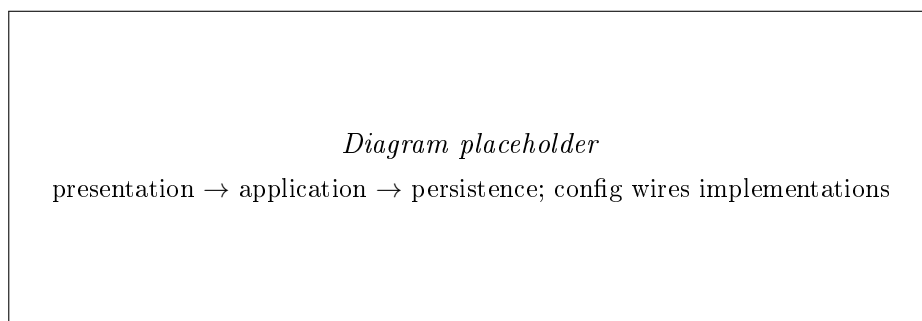
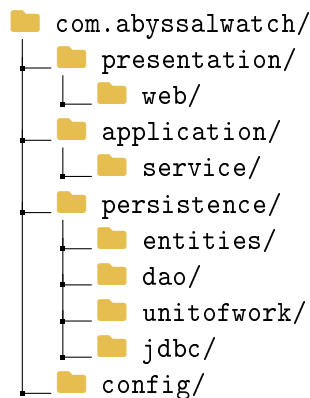


DIAGRAM:
module de-
pendency ar-
rows: pre-
sentation to
application to
persistence;
config wires
implementa-
tions

Figure 1.7: Module dependency direction and configuration wiring (diagram to be added).

Packages are compile-time folders inside one project—the default for student work in this book. **Modules** (Gradle or Maven) add separate compilation units when the codebase grows; they can enforce that `presentation` does not import `persistence.jdbc` directly. Layer rules are the same whether you use packages only or split modules. Figure 1.7 shows the intended dependency arrows between modules.



IN-REF: float
these pack-
age diagrams
to the right,
wrap text on
the left, like
in hexagonal
chapter

Figure 1.8: Baseline package layout for a data-centric three-layer monolith.

Figure 1.8 shows the baseline data-centric package layout, you will notice the top level packages are the layers of the architecture: `presentation`, `application`, and `persistence`.

When the business layer grows, add sub-packages under `application` as in Figure 1.5. When API contracts need stability, add a root `dtos` module (Figure 1.9):

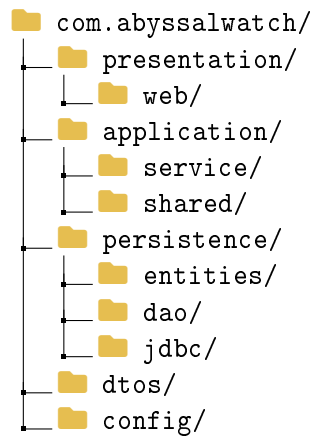


Figure 1.9: Package layout with a separate dtos module.

If requirements expand beyond database access, rename the persistence layer to **infrastructure** to better reflect its more adverse role (Figure 1.10):

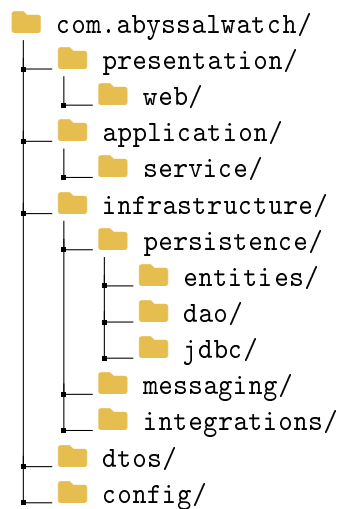


Figure 1.10: Package layout with infrastructure outer ring.

Dependency rules:

- presentation depends on application (and dtos when introduced),
- application depends on persistence DAOs and `UnitOfWork`,
- persistence depends on `persistence.entities` and storage technology,
- presentation must not call persistence directly,
- config wires implementations at startup.

1.8 Assembling Data for Views

Read screens often need joined or aggregated data, not a single entity row. We use `View Mission Kaiju Briefing` as the running read example: for a `MissionId`, the UI shows mission status, linked Kaiju codename and classification, and the origin breach label—data drawn from `mission`, `kaiju`, and `breach`.

Both assembly strategies below produce the same `MissionKaijuBriefingView` (Code snippet 1.9); they differ in *where* joins and composition happen.[3] Figure 1.11 contrasts the two paths.

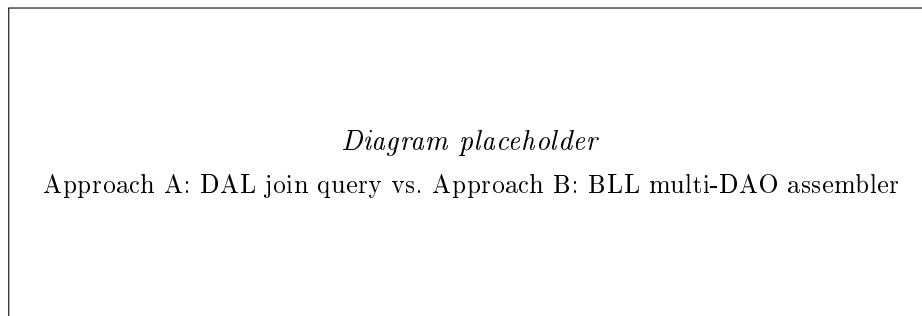


DIAGRAM:
Mission Kaiju
Briefing:
DAL join
query path
vs BLL multi-
DAO assem-
bler path

Figure 1.11: View Mission Kaiju Briefing: data-access join path vs. business-layer assembly path (diagram to be added).

1.8.1 Approach A: Data-Access-Level Assembly

In **Approach A**, Code snippet 1.9 defines the view record and a DAO method that returns a projection, often via a single join query:

```

1 public record MissionKaijuBriefingView(
2     String missionId,
3     String missionStatus,
4     String kaijuCodename,
5     KaijuClassification kaijuClassification,
6     String originBreachLabel
7 ) {}
8
9 public interface MissionBriefingDao
10 {
11
12     Optional<MissionKaijuBriefingView> findBriefing(
13         MissionId missionId
14     );
15
16 }
```

Code snippet 1.9: `MissionKaijuBriefingView` and `MissionBriefingDao`

A `JdbcMissionBriefingDao` implements `MissionBriefingDao` from Code snippet 1.9 with a single JOIN across `mission`, `kaiju`, and `breach`—the SQL lives in `persistence.jdbc`, not in the business layer.

Presentation may call a thin `MissionBriefingQueryService` that delegates to the DAO, or—when the read is purely presentational—use a relaxed path (Section 1.1.4) as in Code snippet 1.10.

```
1 @RestController
2 @RequestMapping("/missions")
3 public class MissionBriefingReadController
4 {
5     private final MissionBriefingDao briefingDao;
6
7     @GetMapping("/{missionId}/briefing")
8     public MissionKaijuBriefingView briefing(
9         @PathVariable String missionId
10    )
11    {
12        return briefingDao.findBriefing(new MissionId(missionId))
13            .orElseThrow();
14    }
15 }
```

Code snippet 1.10: MissionBriefingReadController data-access read path

1.8.2 Approach B: Business-Layer Assembly

In **Approach B**, the business layer loads persistence entities through several DAOs and composes the same view with an **assembler**, keeping policy explicit:

```
1 public final class GetMissionBriefingService
2 {
3     private final MissionDao missionDao;
4     private final KaijuDao kaijuDao;
5     private final BreachDao breachDao;
6     private final MissionBriefingAssembler assembler;
7
8     public MissionKaijuBriefingView getBriefing(
9         MissionId missionId
10    )
11    {
12        MissionEntity mission = missionDao.findById(missionId)
13            .orElseThrow();
14
15        if (!Set.of("Briefed", "Deployed")
16            .contains(mission.getStatus()))
17        {
18            throw new IllegalStateException(
19                "Briefing not available"
20            );
21        }
22
23        KaijuEntity kaiju = kaijuDao.findByMissionId(missionId)
24            .orElseThrow();
25        BreachEntity breach = breachDao.findById(
26            new BreachId(kaiju.getOriginBreachId())
27        ).orElseThrow();
28
29        return assembler.toView(mission, kaiju, breach);
30    }
31 }
```

Code snippet 1.11: GetMissionBriefingService business-layer assembly

Code snippet 1.11 loads entities through the DAOs in Code snippet 1.2 and applies briefing policy before assembly. The same HTTP resource can call the service instead of the DAO directly, as in Code snippet 1.12; the controller only maps the request and response.

```

1  @RestController
2  @RequestMapping("/missions")
3  public class MissionBriefingController
4  {
5      private final GetMissionBriefingService briefingService;
6
7      @GetMapping("/{missionId}/briefing")
8      public MissionKaijuBriefingView briefing(
9          @PathVariable String missionId
10     )
11     {
12         return briefingService.getBriefing(
13             new MissionId(missionId)
14         );
15     }
16 }

```

Code snippet 1.12: MissionBriefingController business-layer read path

1.8.3 Same View, Different Trade-offs

Approach A (Code snippets 1.9 and 1.10) suits stable briefing screens when query efficiency matters: fewer round-trips, database-optimized joins, slimmer business code for read-only flows. Costs: data access absorbs view shapes; DAO APIs multiply as screens evolve.

Approach B (Code snippets 1.11 and 1.12) suits the same view when policy and reuse matter: briefing rules (for example allowed mission states) stay in the service; data access stays entity-oriented. Costs: more DAO calls unless carefully batched; more coordination code—mitigate with `MissionBriefingAssembler`.

For **SCN-03 Multi-Scout Triangulation**, non-trivial fusion rules often push assembly into the business layer with a dedicated **assembler**, even when some sensor reads still come from specialized DAO queries.

1.8.4 SQL Pitfalls When the Business Layer Orchestrates Reads

Approach B in Code snippet 1.11 loads `mission`, `kaiju`, and `breach` through separate DAO calls. With JDBC that usually means **multiple round-trips** to the database—the same class of cost people later call $N+1$ when using ORMs. For hot read screens, prefer Approach A's single join behind Code snippet 1.9 unless policy truly belongs in the service.

Connection and transaction scope: one **Connection** per unit of work (Code snippet 1.5). DAOs must use the connection bound for the current transaction; they must not open silent nested transactions or new connections per call inside the same use case.

Keep SQL behind DAOs: services receive entities or view records from DAO interfaces—not ad-hoc SQL strings. If a course later adopts JPA or Hibernate, lazy-loading and session-boundary mistakes recreate similar read and transaction bugs; this chapter stays with explicit JDBC so those boundaries remain visible.

1.8.5 CQRS Perspective for Read Models

When read complexity dominates, a CQRS-inspired split can separate write entities from read models. That improves read performance but adds maintenance and possible eventual consistency. Treat it as optional evolution, not a day-one default.

1.8.6 Decision Guidance

As a guideline:

- prefer data-access-level assembly when query efficiency matters and view shapes are relatively stable,
- prefer business-layer assembly when policy and cross-channel reuse matter more,
- evaluate CQRS when read scale and complexity dominate.

1.9 Testing Strategy Across Layers

Testing follows the bottom-up boundaries:

- **Data access:** integration tests against a real database—verify SQL, entity mapping, and transactions,
- **Business logic:** tests with fake DAOs that implement the same interfaces as Code snippet 1.2,
- **End-to-end:** a small set of full request-to-response flows through presentation.

For `Confirm First Kaiju Detection`, a focused business-layer test constructs the service from Code snippet 1.7 with `FakeBreachDao`, `FakeKaijuDao`, and an `ImmediateUnitOfWork`, then asserts the service saved the expected `KaijuId`. The fakes implement the same DAO interfaces as Code snippet 1.2—they are not contracts invented by the business core in isolation.

1.10 When Persistence Becomes Infrastructure

1.10.1 Case Trigger from Abyssal Watch

`Confirm First Kaiju Detection` may need to publish a mission alert, not only insert a row. Messaging and integrations belong in the same *outer* ring as storage, still below business logic.

1.10.2 Why Persistence Can Be Too Narrow

The name `persistence` fits when the outer layer is mostly database work. When messaging, APIs, files, or telemetry stabilize alongside storage, `infrastructure` is a clearer name. Dependency direction is unchanged: `presentation -> application -> infrastructure`.

1.10.3 Recommended Evolution Path

1. start with `persistence` while the schema-led stack is mostly database access,
2. rename to `infrastructure` when multiple technical capabilities are stable (Figure 1.10),
3. split by capability: `infrastructure.persistence`, `infrastructure.messaging`, and so on.

The data-centric workflow still begins with storage design; additional adapters extend the outer ring, they do not move schema decisions into presentation.

1.10.4 Mission Alert After Detection

Confirm First Kaiju Detection may need to notify command workflows, not only insert a row. The **decision to alert** stays in the business service; **delivery** belongs in infrastructure (or `persistence.messaging` until you rename the outer ring).

```

1 public interface KaijuDetectionAlertPublisher
2 {
3
4     void publish(KaijuDetectedAlert alert);
5
6 }
7
8 public final class ConfirmKaijuDetectionService
9 {
10     private final BreachDao breachDao;
11     private final KaijuDao kaijuDao;
12     private final UnitOfWork unitOfWork;
13     private final KaijuDetectionAlertPublisher alertPublisher;
14
15     public KaijuId confirmDetection(
16         String codename,
17         KaijuClassification classification,
18         BreachId originBreachId
19     )
20     {
21         return unitOfWork.execute(() ->
22         {
23             breachDao.findById(originBreachId)
24                 .orElseThrow();
25
26             KaijuEntity kaiju = new KaijuEntity();
27             // set fields omitted
28
29             kaijuDao.save(kaiju);
30
31             alertPublisher.publish(
32                 new KaijuDetectedAlert(
33                     new KaijuId(kaiju.getId()),
34                     originBreachId
35                 )
36             );
37
38             return new KaijuId(kaiju.getId());
39         });
40     }
41 }

```

Code snippet 1.13: KaijuDetectionAlertPublisher and extended ConfirmKaijuDetectionService

Code snippet 1.13 extends Code snippet 1.7 with a narrow KaijuDetectionAlertPublisher interface the application depends on, implemented by an adapter in `infrastructure.messaging`. The mechanism can be in-process notification, email, or a message bus—this chapter does not prescribe queues or async plumbing; the adapter is replaceable.

Publishing inside the same unit of work as the database insert can still leave **inconsistent**

outcomes: the row may commit while the alert channel fails afterward, or vice versa if ordering is wrong. Production systems often use an **outbox** table or transactional messaging so the alert is recorded atomically with the write and delivered reliably later. That machinery is out of scope here; the important placement rule remains: the service decides *whether* to alert, infrastructure delivers it. Figure 1.12 shows how the service reaches persistence and the alert adapter.

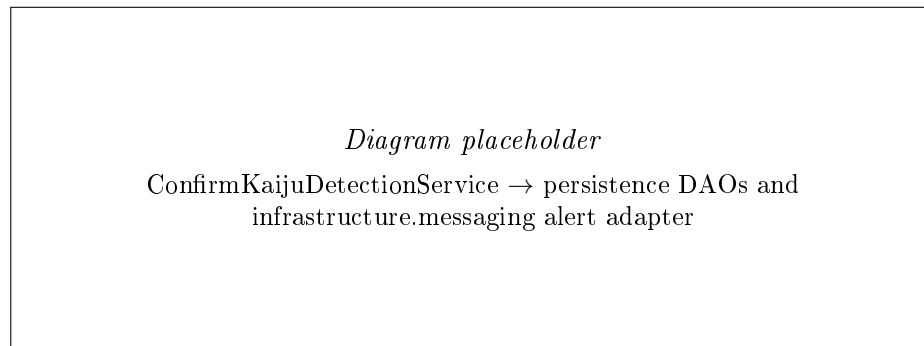


DIAGRAM:
ConfirmKai-
juDetection-
Service to
persistence
DAOs and to
infrastruc-
ture.messaging
alert adapter

Figure 1.12: Service dependencies on persistence DAOs and an infrastructure alert adapter (diagram to be added).

1.10.5 Trade-offs and Guardrails

Avoid turning **infrastructure** into a dumping ground:

- business decisions stay in **application**,
- organize by capability, not framework package names,
- business layer depends on narrow interfaces per capability,
- no **presentation** -> **infrastructure** shortcuts,
- do not put alert orchestration in **presentation** or **DAOs**.

1.11 Schema Change Ripples

1.11.1 A Worked Example: Renaming a Column

Suppose `origin_breach_id` on `kaiju` is renamed to `breach_of_origin_id`. In a data-centric stack the ripple is predictable:

1. migration script updates the table,
2. Code snippet 1.1 and `KaijuDao` SQL in Code snippet 1.4 change,
3. Code snippet 1.7 and any setter calls adjust,
4. the REST field in Code snippet 1.8 may change,
5. Code snippet 1.9 join projection updates if the briefing query exposed the column.

That coupling is a trade-off, not a surprise. When a public API must stay stable through such refactors, introduce or extend **dtos** (Section 1.6). Keep join-heavy mapping in data access; keep policy in services—not in triggers (Section 1.2.3).

1.12 Practical Trade-offs and Common Mistakes

Strengths:[3]

- fast delivery for CRUD-heavy, transactional applications,
- predictable table-to-object mapping and onboarding,
- clear placement rules for new code,
- Transaction Script services over provider-shaped DAOs are easy to teach and ship.[1]

Costs:

- schema changes propagate through layers (Section 1.11),
- swapping storage technology is harder when business code assumes relational shapes,
- rich behavior can accumulate in oversized service classes without sub-package discipline,
- relaxed reads and DAL view projections can hide coupling to schema and screen shapes.

Common mistakes:

- business rules in controllers or UI components,
- orchestration inside DAOs,
- treating relaxed reads as the default for every query,
- presentation calling data access directly on writes or policy-heavy reads (contrast Code snippets 1.8 and 1.10),
- logic in stored procedures that should live in the business layer,
- exposing raw persistence entities in public APIs without a DTO evolution path,
- alert orchestration in presentation or DAOs instead of the placement in Code snippet 1.13,
- closing the shared `Connection` inside a DAO while a `UnitOfWork` is still active (Code snippets 1.4 and 1.5),
- SQL strings or `PreparedStatement` usage in services instead of DAOs,
- untyped request bodies (for example a generic `Map`) at the presentation boundary without shape validation.

1.12.1 Layered and Hexagonal: Practical Relationship

Data-centric layering and hexagonal architecture address the same problem—separating behavior from technology—with different anchors. Classic three-layer starts from the **database schema** and builds upward; hexagonal starts from **use-case behavior** and plugs technology in through ports. Neither is universally “correct.”

When integration churn, testing friction, or the need for core-owned contracts grows, teams can evolve selected areas toward ports and adapters (Chapter ??). That is a refinement path, not an admission that layering was wrong for systems that genuinely center on accurate record capture and storage.

1.12.2 Closing Thoughts

This chapter taught **data-centric three-layered architecture** inside a logical monolith: the relational schema is the anchor, and software grows upward in predictable layers. The journey was intentionally bottom-up: design tables and write SQL, implement JDBC DAOs and persistence entities (Code snippets 1.1–1.6), add Transaction Script services (Code snippet 1.7) with explicit transaction boundaries, expose thin REST adapters (Code snippet 1.8), then extend the shape when complexity appears—shared helpers and assemblers, optional **dtos** (Figure 1.9), infrastructure for alerts (Code snippet 1.13), read assembly (Code snippets 1.9–1.12), and controlled relaxed reads where policy does not apply. Sections on DTO modules, infrastructure renaming, and advanced read strategies are **evolution** topics, not day-one requirements.

For your course work, hand-written SQL stays in **persistence**; layers exist so that SQL, HTTP details, and business policy do not collapse into one place. When provider-shaped DAOs, testing friction, or integration churn grow, hexagonal architecture offers a behavior-first refinement (Section 1.12.1)—not because layering failed, but because the problem outgrew schema-led defaults. For CRUD-heavy, transactional systems that genuinely center on accurate record capture, the classic three layers remain a strong, teachable default.

Bibliography

- [1] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.
- [2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. 2nd ed. Addison-Wesley Professional, 2018. ISBN: 978-0134757599.
- [3] Microsoft Corporation. *Layered Application*. Patterns & Practices. 2003. URL: [https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff650258\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff650258(v=pandp.10)) (visited on 06/04/2026).
- [4] Microsoft Corporation. *Using a Three-Tier Architecture Model*. 2026. URL: <https://learn.microsoft.com/en-us/windows/win32/cos-sdk/using-a-three-tier-architecture-model> (visited on 06/04/2026).